



Weather App — Récapitulatif de Projet

Ce projet consiste à développer une application météorologique complète, de la géolocalisation navigateur jusqu'à la persistance des données en base SQLite, en passant par l'intégration d'une API météo externe. Il illustre la maîtrise de Node.js, Express, des API Web et des bonnes pratiques de sécurité.

Vue d'ensemble du projet

L'application **WeatherApp** est construite en plusieurs étapes progressives, chacune ajoutant une nouvelle fonctionnalité au projet. Voici les grandes parties couvertes :

API Geolocation

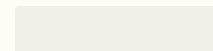
Envoi des données au serveur Node.js via une requête HTTP POST

POST Request avec fetch()

Envoi des données au serveur Node.js via une requête HTTP POST

Stockage en tableau & fichier JSON

Persistence des coordonnées dans un tableau serveur puis dans un fichier database.txt



Base de données SQLite

Enregistrement structuré des données dans une base weatherApp.db

API WeatherAPI & sécurité

Intégration de données météo et protection de la clé API via .env

1. Utiliser l'API Web « Geolocation » côté navigateur

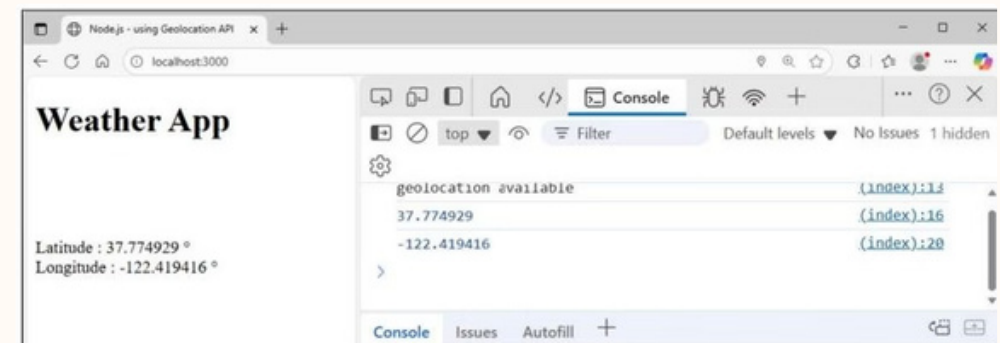
L'API Web « Geolocation » permet de connaître la géolocalisation d'un utilisateur. Elle est accessible via un appel à la propriété `navigator.geolocation` du navigateur. Pour des raisons de confidentialité, le navigateur va demander à l'utilisateur l'autorisation d'accéder à ses données de localisation.

Si l'utilisateur accepte de partager les données de localisation, le navigateur utilisera la meilleure fonctionnalité disponible sur l'appareil pour accéder à ces informations (GPS, données de connexion à Internet...).

Étapes réalisées

- Création du projet Node.js **WeatherApp**
- Installation d'**Express** et création du dossier `public`
- Ajout du fichier `index.html` dans `public`
- Configuration d'Express sur le port **3000**
- Vérification de la disponibilité de l'API Geolocation
- Récupération des coordonnées GPS avec `getCurrentPosition()`
- Affichage de la latitude et longitude dans la page Web
-

Résultat obtenu



La page affiche **Latitude : 37.774929 °** et **Longitude : -122.419416 °**, confirmant la récupération correcte des coordonnées GPS via l'API Geolocation.

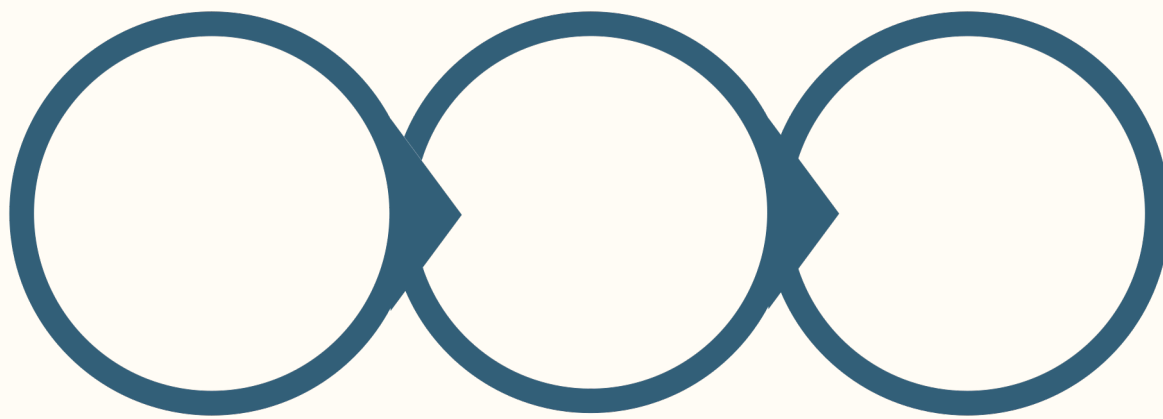
- Nous utilisons ici une **Arrow function** : on exécute les instructions prévues seulement une fois que les coordonnées sont récupérées. L'instruction `document.getElementById()` renvoie l'objet dont la propriété `id` correspond à la chaîne de caractères spécifiée en paramètre.

2. POST Request avec fetch()

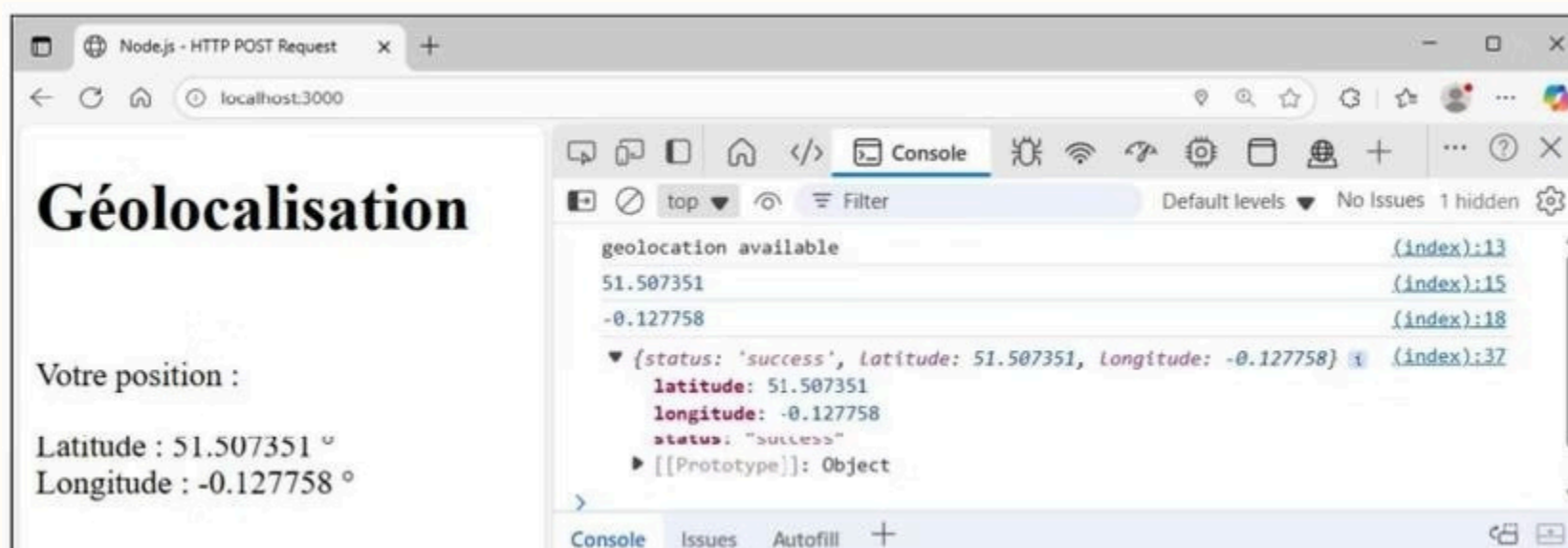
Ici, nous utilisons la méthode `POST` pour que notre client (le fichier `index.html` interprété par un navigateur) puisse envoyer des données au format JSON à notre serveur Node.js.

La première étape consiste à créer sur notre serveur une nouvelle route dérivée de la méthode `http` « `POST` ». Nous allons attacher cette route à une instance du Framework **Express**.

- ❏ Le « **routage** » fait référence à la façon dont les points de terminaison (URI) d'une application répondent aux demandes des clients.



La constante `options` passée en paramètre de `fetch()` indique au serveur la méthode utilisée (ici `POST`), le type de données envoyées (JSON) et les données dans le paramètre `body` de la requête. On utilise une **fonction asynchrone** pour attendre la réponse et une autre pour attendre la mise en forme des données au format JSON.



- ❏ À chaque changement dans votre code, vous devez redémarrer votre serveur Node.js. Utilisez le module **nodemon** (`npm install -g nodemon`) pour que les modifications soient automatiquement prises en compte : `nodemon index.js`.

3. Stocker les données dans un tableau

L'objectif est d'ajouter sur la page `index.html` un bouton qui déclenche l'envoi des coordonnées géographiques au serveur Node.js, puis d'ajouter ces données à un tableau de coordonnées.

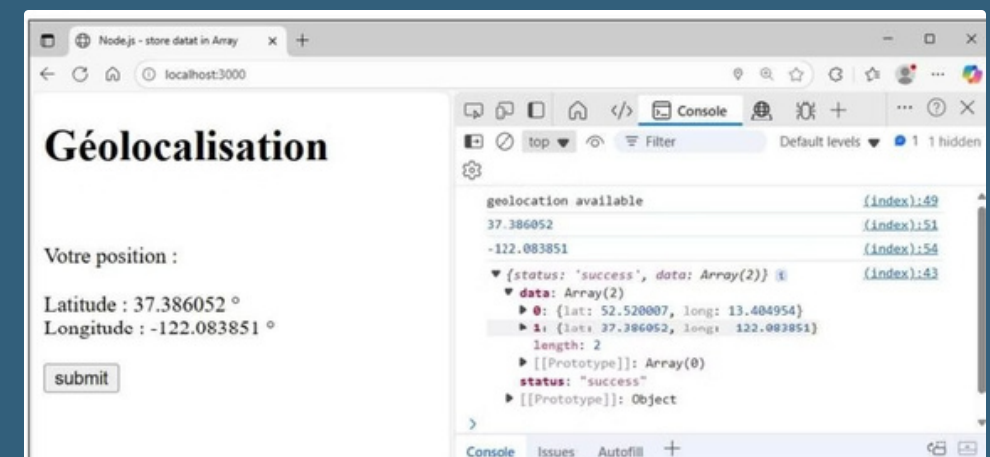
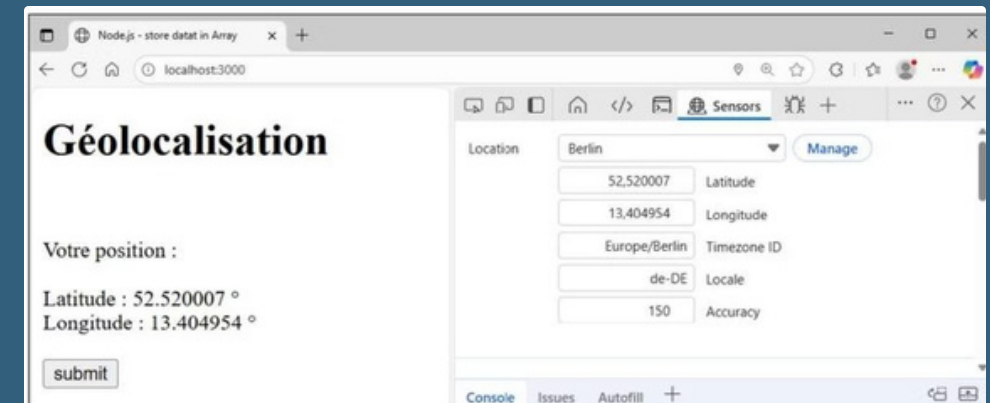
Côté serveur — `index.js`

Un tableau `dataArray` est créé pour stocker les informations (une ligne par request). À chaque requête POST, les données sont ajoutées au tableau avec `dataArray.push(data)`, puis renvoyées au client.

```
[nodemon] starting `node index.js`  
listening at 3000  
I got a request  
{ lat: 52.520007, long: 13.404954 }  
[ { lat: 52.520007, long: 13.404954 } ]  
I got a request  
{ lat: 37.386052, long: -122.083851 }  
[  
  { lat: 52.520007, long: 13.404954 },  
  { lat: 37.386052, long: -122.083851 }  
]
```

Test avec le mode développeur

Pour tester, le mode développeur du navigateur est utilisé. L'onglet **Sensors** permet de modifier la géolocalisation avant chaque envoi vers le serveur.



- ✔ Nous avons réussi à stocker les données sur notre serveur Node.js ! Le tableau `dataArray` accumule les coordonnées de chaque requête et les renvoie au client dans la réponse JSON.

4. Enregistrer les données au format JSON dans un fichier texte

Ils'agit ici de s'assurer de la **persistance des données**, en enregistrant les coordonnées dans un fichier nommé `database.txt`. Nous utilisons le module **File system**, inclus par défaut dans Node.js:

```
const fs = require("fs");
```

saveToDatabase(data)

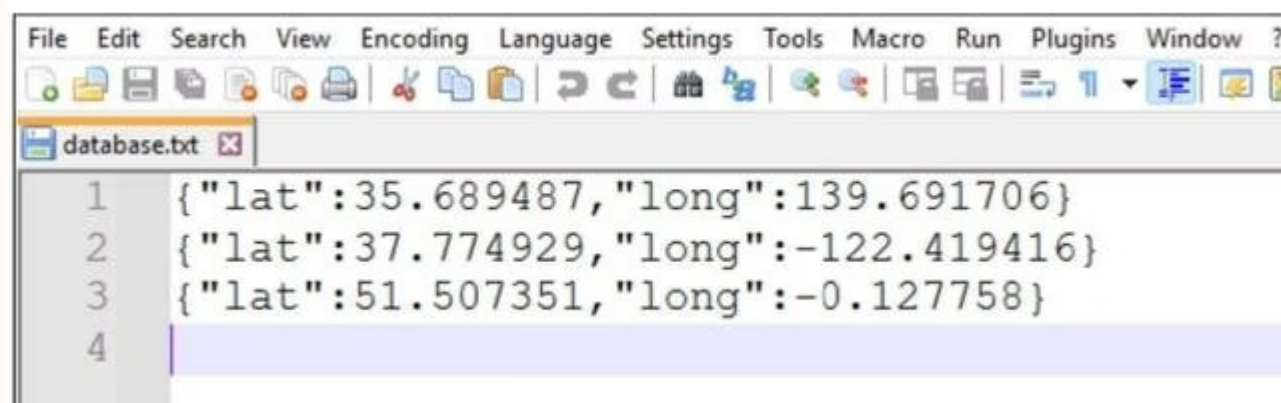
Convertit les données en chaîne JSON avec `JSON.stringify()` puis les ajoute de manière synchrone dans `database.txt` avec `appendFileSync()`. Si le fichier n'existe pas, il est créé automatiquement. Un saut de ligne `\n` est ajouté entre chaque entrée.

getDatabase()

Lit le fichier `database.txt` avec `readFileSync()`, découpe les lignes avec `split("\n")`, parse chaque ligne avec `JSON.parse()` et retourne un tableau contenant toutes les données enregistrées.

clearDatabase()

Remet à zéro les données en écrasant le fichier avec une chaîne vide via `fs.writeFileSync("database.txt", "")`. Utile pour repartir d'une base propre lors des tests.



Le fichier `database.txt` contient bien les coordonnées enregistrées ligne par ligne au format JSON. La route `/api` est modifiée pour appeler `saveToDatabase(data)` puis `getDatabase()` et renvoyer la totalité des données comme réponse.

5. Enregistrer les données dans une base de données SQLite

Nous utilisons maintenant une base de données **SQLite** pour enregistrer nos coordonnées ainsi que d'autres informations. Le projet **WeatherAppSQLite** reprend le code de WeatherApp avec le package `sqlite3` (`npm install sqlite3`).

Structure de la table `geoloc`

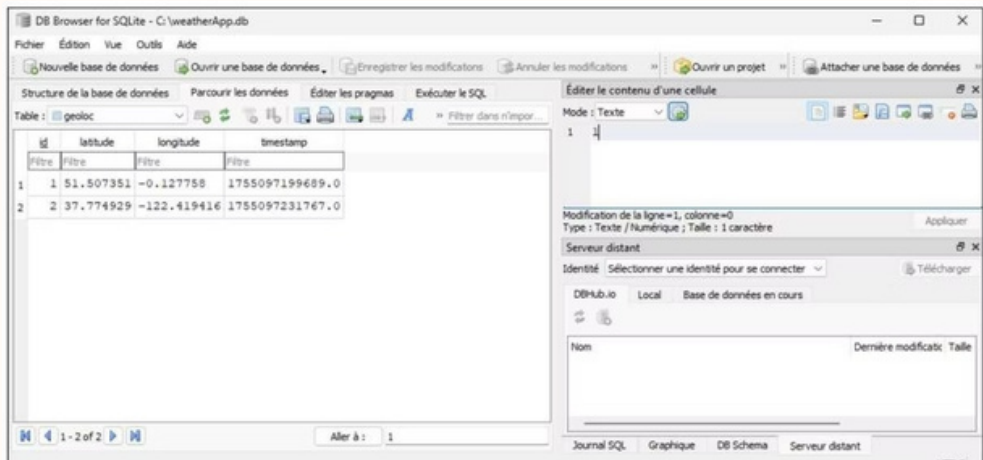
Champ	Type	Description
<code>id</code>	INTEGER PRIMARY KEY TEXT NOT NULL	Identifiant unique
<code>latitude</code>	TEXT NOT NULL	Latitude GPS
<code>longitude</code>	REAL NOT NULL	Longitude GPS
<code>timestamp</code>		Date et heure de création

- La table n'est créée que lors de la première exécution grâce à `CREATE TABLE IF NOT EXISTS`.

Fonctions principales

- `Insérer(lat, long, timestamp)` : prépare et exécute un `INSERT INTO geoloc`
- `Afficher()` : affiche tous les enregistrements dans la console via `SELECT`
- `databaseInsert(data)` : fonction **asynchrone** qui attend la fin de l'insertion avant d'afficher

Résultat dans DB Browser for SQLite



Les données sont bien enregistrées dans la base `weatherApp.db`, visualisables avec **DB Browser for SQLite**.

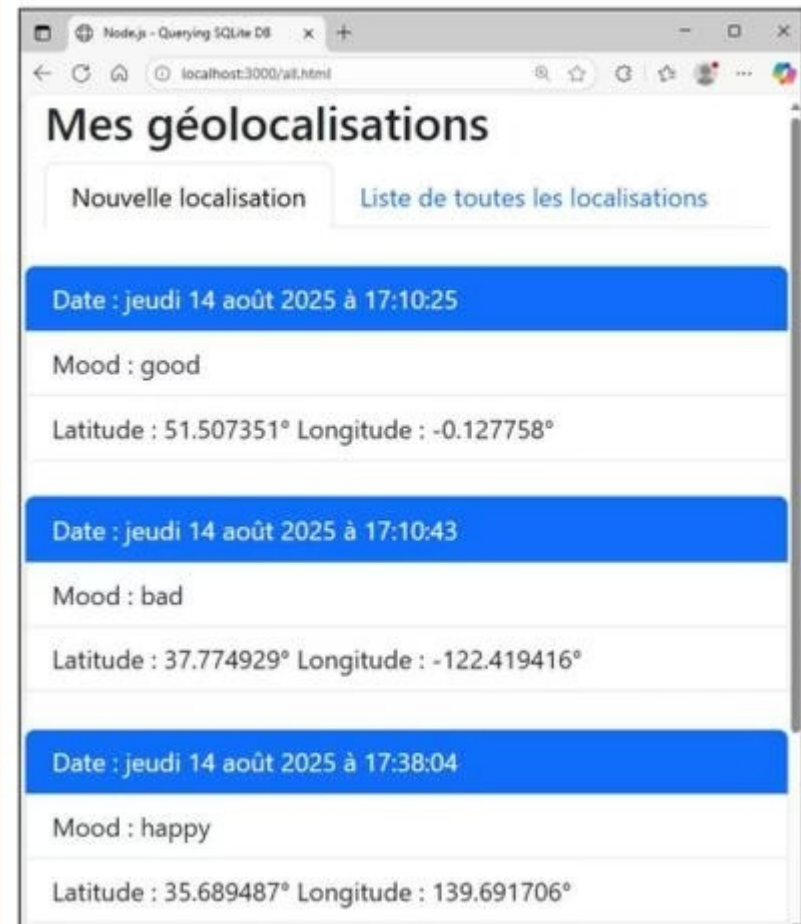
TRAVAIL À FAIRE

- Ajouter un champ texte « **Mood** » lors de la création de la table `geoloc`
- Permettre à l'utilisateur de saisir son humeur sur `index.html` et l'enregistrer dans la base
- Créer une page `all.html` affichant l'ensemble des enregistrements de `weatherApp.db`
- Ajouter un menu de navigation entre `index.html` et `all.html`
- Utiliser Bootstrap pour rendre le site « responsive »

Page `index.html`



Page `all.html`



6 & 7. Requêtes SQLite & Intégration de l'API WeatherAPI

Nous utilisons l'API **WeatherAPI** pour récupérer des données géographiques (ville, pays) ainsi que des données météorologiques (le temps qu'il fait, la température, la qualité de l'air et la concentration en particules fines).

Données récupérées via WeatherAPI

- **Ville** : `json.location.name`
- **Pays** : `json.location.country`
- **Conditions météo** : `json.current.condition.text`
- **Température** : `json.current.temp_c`
- **Index qualité de l'air (UK DEFRA)** : `json.current.air_quality["gb-defra-index"]`
- **Particules fines PM2,5** : `json.current.air_quality.pm2_5`

La nouvelle route `/weather/:lat/:long` reçoit deux paramètres (latitude et longitude) et interroge l'API WeatherAPI avec la clé `apiKey`. Les opérations sont encadrées par un `try-catch` pour gérer les erreurs d'indisponibilité de l'API.

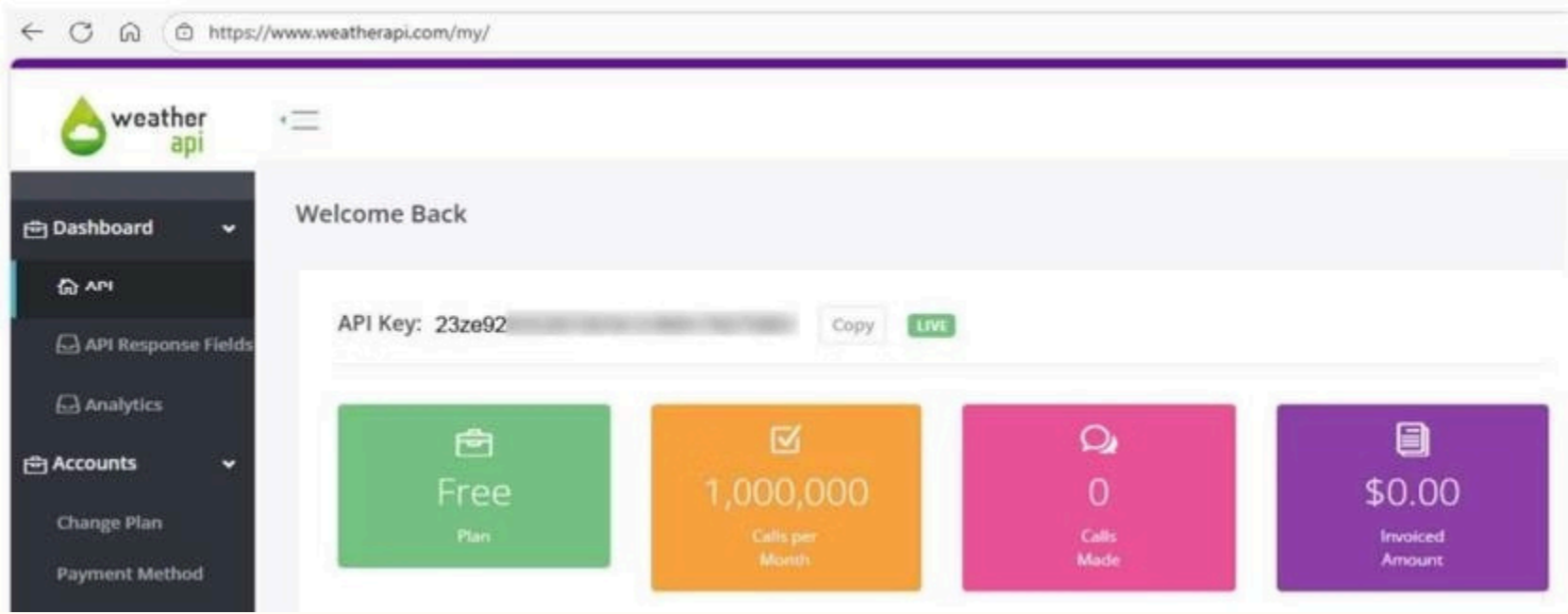
Résultat final — page `all.html`



Chaque entrée affiche la date, le mood, les conditions météo, la température et le niveau de pollution (Faible / Modéré / Élevé / Très élevé selon l'index UK DEFRA).

TRAVAIL À FAIRE

- Modifier `app.post('/api')`, `databaseInsert()`, `Afficher()`, `Inserer()` et `db.serialize()` pour enregistrer toutes les données affichées dans `index.html`
- Modifier l'affichage de l'index de qualité de l'air pour afficher un résultat plus parlant (niveau faible, modéré, élevé, très élevé) — voir <https://uk-air.defra.gov.uk/air-pollution/daqj>
- Modifier la fonction `GetData()` de la page `all.html` pour afficher toutes les données enregistrées dans la base

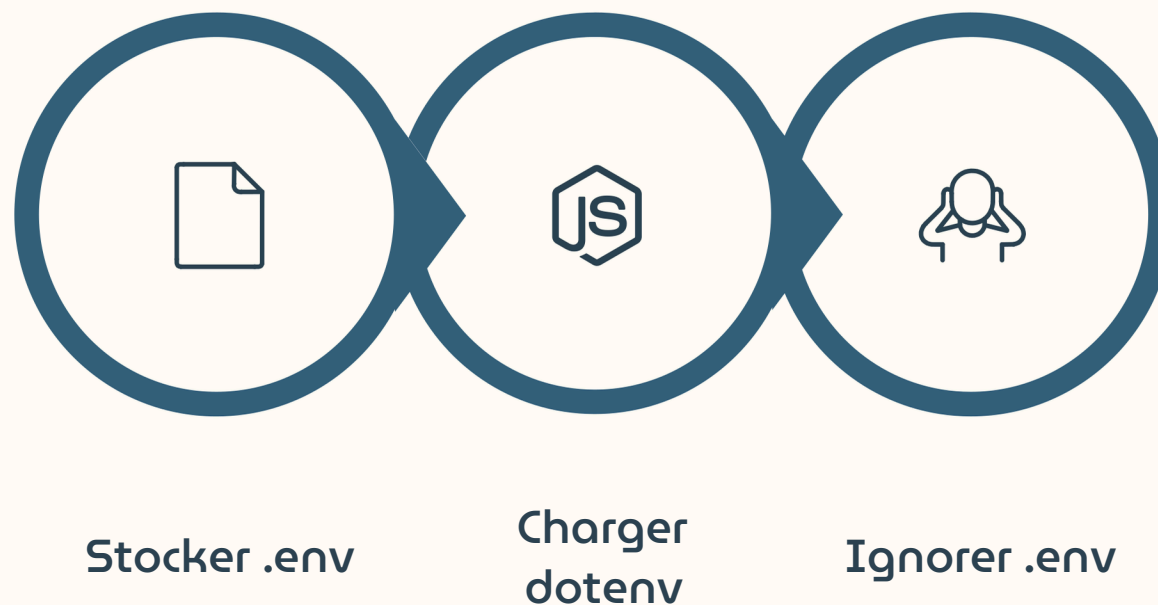


Connectez-vous sur <https://www.weatherapi.com/>, créez un compte et récupérez votre **API Key** dans le tableau de bord (« Dashboard »). Cette clé sera nécessaire pour utiliser l'API dans notre application.

8. Cacher une « API Key » avec les variables d'environnement

Dans notre projet, la clé qui permet d'accéder à l'API weatherapi.com est clairement visible dans notre code. Nous allons utiliser une **variable d'environnement** stockée dans un fichier .env pour cacher notre clé.

Commencez par installer la bibliothèque **dotenv** en utilisant la commande suivante : `npm install dotenv`.



TRAVAIL À FAIRE

1 Étudier la documentation dotenv

Consulter la page <https://www.npmjs.com/package/dotenv> et cacher votre API key dans un fichier .env situé à la racine de votre projet.

2 Modifier index.js

Modifier le fichier index.js pour utiliser la variable d'environnement à la place de la clé API en clair dans le code.

3 Publier sur QitHub

Enregistrer votre projet dans un repository Github **sans prendre en compte** le fichier .env et les bibliothèques installées (via .gitignore).

⚠ Ne jamais publier votre fichier .env sur GitHub ! Ajoutez-le impérativement à votre .gitignore pour protéger votre clé API et éviter tout usage non autorisé.

Récapitulatif des compétences acquises

Ce projet **WeatherApp** couvre l'ensemble du cycle de développement d'une application web full-stack avec Node.js, de la récupération de données navigateur jusqu'à la sécurisation des accès API.



API Web Navigateur

Utilisation de l'API **Geolocation**, gestion des permissions, Arrow functions et fonctions asynchrones (**async/await**)



Node.js & Express

Création de routes **GET** et **POST**, middleware **express.json()**, serveur statique, routage paramétré



Persistence des données

Stockage en tableau serveur, fichier texte **JSON** avec le module **fs**, base de données **SQLite** avec requêtes **SQL**



Intégration API externe

Appel à **WeatherAPI** avec **fetch()**, traitement des données **JSON**, affichage de météo et qualité de l'air



Sécurité & Bonnes pratiques

Protection des clés API avec **dotenv** et **.env**, gestion des erreurs avec **try-catch**, publication GitHub avec **.gitignore**



Front-end responsive

Interface utilisateur avec **Bootstrap**, navigation multi-pages, affichage dynamique du DOM avec **document.getElementById()**